

PolyTaste Project Explanation and Viva Guide

Prepared for the project discussion on 7 October 2026. Code, datasets and test evidence were inspected on 6 October 2026.

What the project does

PolyTaste combines recommendations across music, anime, movies, Chennai tourist places, restaurants and Myntra fashion. It collects explicit signals such as likes and ratings, and implicit signals such as listening history or product views. Each domain has its own recommendation method. A shared taste profile and manually defined genre crosswalks let signals from one domain influence another.

The contribution is the integration of these domains, preference signals, explainable ranking and feedback into one application. The crosswalks are hypotheses about taste transfer. Their usefulness must be evaluated rather than assumed.

Your opening explanation

“My project is PolyTaste, a cross-domain recommendation application. The frontend is React with TypeScript, and the backend is FastAPI. SQLAlchemy stores account connections, histories and feedback in a relational database. I use local SQLite for the demonstration and the code also supports PostgreSQL. Anime descriptions are converted into embeddings and compressed by an autoencoder. Movies use TF-IDF and cosine similarity. Music has a genre-profile recommender and a separate experimental DNN trained from listening sessions. Tourist places and dining use category mappings and feedback. Fashion uses browsing signals, text similarity, price fit and cross-domain mappings. I tested the application behavior separately from recommendation quality, and I will state the limitations of each evaluation.”

The terminology to use

There are no implemented graph neural networks in the inspected source. A DNN is a feed-forward neural network; an autoencoder learns to reconstruct its input. A GNN would require a graph representation and message passing between nodes. A co-occurrence matrix and a dictionary of genre mappings do not make the present implementation a GNN.

The current implementation is a hybrid of neural representation learning, content similarity and deterministic rules. It is not a demonstrated multi-user collaborative filtering system.

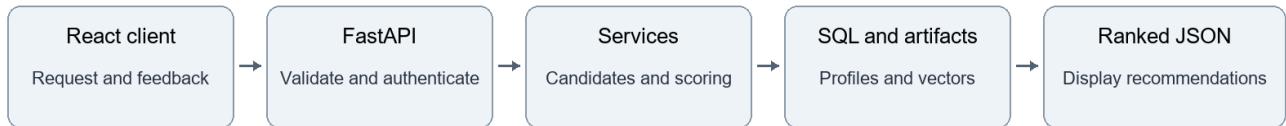
How to use this guide

First understand the pipeline and data, then the individual algorithms, then what the tests measure. The later pages provide answers to likely viva questions. The two accompanying text files contain complete prompts for generating the formal report and conducting a rigorous testing pass.

Architecture and Technology Stack

End to end flow

The React client sends JSON requests to FastAPI. Authenticated requests carry a signed session cookie. Routers validate inputs and call service functions. Services retrieve catalog metadata, user signals and feedback from SQL or precomputed files, generate candidates, score them and return JSON recommendations. The browser displays the results and sends later feedback back to the API.



Offline scripts prepare features and models before online ranking

Technology responsibilities

Layer	Technology	Role in this project
User interface	React 19 and TypeScript	Pages, components, state and API requests
Frontend tooling	Vite 8 and Tailwind CSS 3	Development server, build and styling
UI support	Motion and Framer Motion, Radix, Lucide, Recharts and Visx	Animation, controls, icons and charts
API server	Python, FastAPI, Uvicorn and Pydantic	HTTP routing, validation and serving
Persistence	SQLAlchemy, SQLite or PostgreSQL	Relational tables and database sessions
Numerical ranking	NumPy and scikit-learn	Embeddings, TF-IDF, SVD and similarity
Offline learning	PyTorch and sentence- transformers	DNN training and anime feature learning
Integrations	Requests, Google OAuth, Spotify and AniList APIs	Identity and optional account data
Additional content	TMDB, Kitsu or Jikan, YouTube and RSS	Catalog and supporting content
Background tasks	APScheduler	Spotify synchronization every 30 minutes
Browser integration	JavaScript Manifest V3 extension	Authorized visible Myntra page activity
Verification	pytest, Node test runner, TypeScript and Vite	Backend, extension and frontend checks

The local test environment used Python 3.14.5, pytest 9.1.1 and PyTorch 2.13.0 CPU. Document tooling is separate from the application stack.

Training is an offline script operation. Normal requests generally load precomputed arrays and rank candidates. The optional Spotify DNN route still imports PyTorch at request time, although the production requirements file omits PyTorch.

Source: [main.py](#); [frontend/package.json](#); [requirements files](#); [services/spotify_scheduler.py](#); [extension/manifest.json](#).

Database and Storage Design

SQLAlchemy maps Python classes to relational tables. DATABASE_URL selects the backend; without it, SQLite uses SQLITE_PATH or spotify_tokens.db. LOCAL_SETUP.md specifies local_polytaste.db for the local demo. PostgreSQL support comes from a configured connection URL and psycopg2. Database rows store operational data; JSON catalogs and pickle/model artifacts store offline datasets and learned representations.

Local database inventory

These are counts from the read-only local_polytaste.db snapshot before report tests.

Table or group	Rows	Purpose
users	4	Google identity and display preferences
spotify_users	2	Spotify connection, tokens and sync settings
anilist_users	2	AniList connection and account metadata
spotify_play_events	154	Incremental live listening events
spotify_import_profiles	0	Profiles derived from exported history
user_likes	0	Explicit domain item likes
movies	255	Movie metadata and shared personal_rating field
tourist_spots	65	Chennai place catalog
dining_spots	872	Chennai restaurants and cafes
user_spot_feedback	1	Place likes or dislikes
user_dining_feedback	0	Dining likes or dislikes
recommendation_feedback	665	Shown, liked and disliked recommendations
myntra_products	510	Seeded and observed fashion products
myntra_events	29	Collected fashion activity
myntra_connections	2	Collection permissions and enabled state
myntra_profiles and myntra_feedback	0 each	Stored fashion profiles and feedback

Identity and constraints

Google login establishes an internal User.id. Provider identities remain separate: google_sub, Spotify account ID and AniList ID. user_likes has a unique user/module/item constraint; spotify_play_events has a unique user/played_at constraint; Myntra event_id and product_id support deduplication.

Two design limitations matter: tourist/dining feedback declares a foreign key to users.google_sub although session code normally uses internal user IDs, and movies.personal_rating belongs to the catalog rather than a separate per-user rating table. SQLite success does not prove PostgreSQL foreign-key compatibility. A production design should standardize user keys and introduce per-user ratings.

Anime has no dedicated SQL catalog table in this design; its metadata and embeddings are loaded from files. JSON encoded in text columns is a serialization choice, not a separate NoSQL database.

Source: database.py; models/base.py; models/myntra.py; qa/inventory.json.

Dataset Inventory and Provenance

What real world data means here

Your friend's Spotify export is real recorded behavior from one participant. Public movie/anime metadata and OpenStreetMap places are real item information. Synthetic ratings and UI mock recommendations are different forms of data and must be identified separately.

Dataset or artifact	Inspected size	Interpretation
Spotify audio JSON files	7 files and 28,407 events	One friend's exported listening history
Artist and track valid Spotify events	28,336	Records retained by the current metadata filters
Spotify unique track URIs	3,846	Distinct tracks in the supplied export
Spotify artists	1,537	495 have at least five play records
Anime source CSV	15,000 rows	Local anime metadata; original download provenance needs documentation
Anime usable text and dense cache	14,979 rows by 384 features	Text accepted by the training filter
Anime serving embeddings	14,976 by 32 features	Final ID-keyed latent artifact
Movie serving matrix	255 by 5,000 features	TF-IDF over overview and genres
Tourist and dining JSON	65 and 872 entries	Chennai catalog derived from OSM or curated exports
Spotify serving track embeddings	1,850 by 32 features	Experimental DNN artifact; no genre labels
Myntra seed CSV	15,000 source rows	Local catalog seed, not personal user behavior
Additional fashion catalog CSV	12,491 rows	Separate catalog file, not all currently loaded

The valid Spotify events cover 29 June 2020 through 9 April 2026, with approximately 860.82 hours of recorded playback. These are activity counts, not independent participants or guaranteed complete track listens. The export also contains video files; the current audio pipelines ignore those files.

The 14,979 versus 14,976 anime difference concerns usable text rows versus the final ID-keyed artifact. Embedding generation also rejects invalid IDs and a dictionary can collapse repeated IDs. Do not invent the exact reason for each missing row without a row-level audit.

Why the two databases differ

spotify_tokens.db contains older data: 251 rated movies, 230 ratings at least seven, two imported Spotify profiles and four tourist feedback rows. local_polytaste.db has no movie personal ratings and no imported Spotify profiles. The cached movie artifact still includes 251 ratings. The project contains a synthetic-rating generator and no provenance manifest proving those cached ratings are genuine IMDb exports.

Source: qa/inventory.json; scripts/generate_synthetic_ratings.py; scripts/import_imdb_ratings.py; dataset and artifact files.

Spotify Data Format for the Demonstration

Keep the original JSON format

No conversion is needed for the current importer. Put the Audio files in one folder and retain their names, such as Streaming_History_Audio_2024.json. Each file should contain a JSON array of event objects. A CSV can help human inspection, but it is not the current import format.

This is an illustrative object with invented track and artist names, not an extracted private record.

```
[
  {
    "ts": "2025-01-15T10:30:00Z",
    "ms_played": 180000,
    "master_metadata_track_name": "Example Track",
    "master_metadata_album_artist_name": "Example Artist",
    "spotify_track_uri": "spotify:track:EXAMPLE_ID",
    "skipped": false
  }
]
```

Field	Meaning	Use
ts	UTC playback timestamp	Ordering and session boundaries
ms_played	Actual recorded playback in milliseconds	Listening affinity and skip proxy
master_metadata_track_name	Track title	Display and DNN validity filter
master_metadata_album_artist_name	Artist name	Artist aggregation and API enrichment
spotify_track_uri	Track identifier	Session co-occurrence and track embeddings
skipped	Exported skip flag	Combined with the 30-second rule

The artist-profile importer requires a nonempty artist name. The DNN parser also requires a track URI and title and reads the timestamp. The current code is not a complete strict JSON-schema validator, so a new dataset should be audited before importing.

Optional research format

For a future multi-user study, derive an event table with participant_id, track_id, artist, timestamp_utc, ms_played, skipped, source_file and source_row. Keep artist genres in a separate enriched lookup. Preserve the raw export privately and store checksums in a provenance manifest. Do not combine different friends into one unidentified listening timeline.

Show your professor an anonymized sample, the field definitions and aggregate counts. Get your friend's permission for the academic use and remove IP, device and unrelated private fields from presentation material. The importer ignores these fields, but the original export still contains them.

Source: scripts/import_spotify_history.py; scripts/train_spotify_history.py; qa/inventory.json.

Spotify Genre Profile Pipeline

This path converts exported listening behavior into explainable genre weights. It is separate from the DNN training path.

1. Read and merge all Audio JSON files. Drop records whose artist is null. The supplied data retain 28,336 of 28,407 raw events.
2. Aggregate by artist: play count, total milliseconds and skip count. An explicit skip or less than 30 seconds is treated as a skip.
3. Keep artists with at least five records. In this export, 495 artists survive that rule.
4. Compute $\text{completion_rate} = 1 - \text{skip_count} / \text{play_count}$. This is a non-skip ratio proxy; the code does not compare playback with the full song duration.
5. Compute $\text{affinity} = \text{total_ms_played} \times \text{completion_rate}$ and select the top 200 artists.
6. Obtain an app-level Spotify token, search artist IDs and fetch artist genres in batches. Normalize genre names to broader categories.
7. For each genre, sum the affinities of artists carrying that genre. Persist the genre dictionary and top-artist summary in `spotify_import_profiles`.

A worked example

Suppose an artist has ten records, 1,800,000 total milliseconds and two skips. `completion_rate` is 0.8 and affinity is 1,440,000. If that artist is tagged rock and indie, the current code adds 1,440,000 to each genre. The same affinity can contribute to multiple genre buckets.

Live recommendation path

The primary `/spotify/recommendations` route builds a profile from live top artists, track artists and recent history, with fallback paths. Candidate tracks come from genre or artist searches. The normal genre score is the sum of profile weights matching the candidate's artist genres. Feedback-aware reranking follows. If genre scoring finds no scored candidates, a candidate-order fallback produces heuristic scores.

Current integration limits

The `/spotify/import-history` endpoint accepts a folder on the backend machine. It is not a file-upload API. A folder on your laptop cannot be read by a remote server unless transferred there separately. The command-line dry-run skips storage but still performs API enrichment.

The shared profile gathers Spotify only when a token is passed. Its imported-profile fallback sits inside that token-dependent flow; import alone does not guarantee cross-domain activation without a Spotify connection. Imported affinities also retain millisecond-scale weights, unlike live rank weights, which can distort relative domain influence.

The inventory found 169 duplicate track/timestamp pairs. The existing import/training parsers do not deduplicate them. Artist-name search can be ambiguous and genre enrichment can be empty or fail. These are data-quality and integration issues to report.

Source: `scripts/import_spotify_history.py`; `routers/spotify_import.py`; `routers/spotify.py`; `services/taste_profile.py`.

Spotify Session DNN Pipeline

How the model is trained

`scripts/train_spotify_history.py` sorts valid track records by timestamp and starts a new session when the gap exceeds 30 minutes. It keeps sessions with more than one event. It retains tracks occurring at least twice within these retained sessions, builds a sparse co-occurrence matrix using a context window of five tracks on either side, and applies TruncatedSVD to produce 32-dimensional vectors. The vectors are normalized by their lengths.

Adjacent track pairs sampled from sessions are positive examples. Random distinct track pairs are negative examples. These are co-listening proxy labels, not explicit declarations that a user likes one track over another. Random negatives can include genuinely related tracks.

Two 32-dimensional embeddings are concatenated into a 64-dimensional input. The history trainer configures the network as 64 to 64 to 32 to 1, with ReLU hidden activations and a sigmoid output. It uses binary cross-entropy and Adam with learning rate 0.005 for a default 20 epochs. It targets up to 20,000 sampled pairs, although the actual number depends on available positives.

The artifacts are `spotify_model.pth` and `track_embeddings.json`. The inspected embedding artifact contains 1,850 tracks. Existing artifacts alone do not establish the exact source-export version or original training time; a training manifest should record these.

What inference does

The optional `/spotify/recommend/{track_id}` route retrieves a seed embedding, scores it with each other stored track embedding and sorts by `similarity_score`. It loads the DNN from disk and requires the development PyTorch dependency. Normal genre-profile recommendations do not depend on this DNN.

The `models/spotify_dnn.py` class defaults still describe a ten-feature audio-feature input. The history training script overrides those defaults to 64 input features and a 64-unit hidden layer. The older `scripts/train_spotify.py` uses synthetic genre-cluster feature pairs. Do not mix results from these two training approaches.

What is not measured

The history trainer prints loss and accuracy on the same pairs used for training. It has no held-out validation/test split, so those numbers cannot establish generalization. Python random sampling and Torch initialization are not fully seeded in that script. The saved genre-overlap evaluator cannot assess this DNN because the track artifact contains no genres.

A defensible future test must split whole sessions chronologically before building co-occurrence/SVD, fit everything on training only, and evaluate untouched test pairs or rankings against co-occurrence, SVD-cosine, popularity and random baselines. A single friend's history supports a personal case study.

Source: `models/spotify_dnn.py`; `scripts/train_spotify_history.py`; `scripts/train_spotify.py`; `routers/spotify.py`; `qa/verified_metrics.json`.

Anime Representation Learning and Inference

Offline feature pipeline

The training script reads synopsis, genres and themes from `top_15000_anime.csv`. It drops records with neither synopsis nor genres. `all-MiniLM-L6-v2` converts the combined text into 384-dimensional sentence embeddings. These dense vectors are the autoencoder inputs; the older README's 1,000-dimensional TF-IDF description does not match the current training script.

The encoder is 384 to 128 to 32; the decoder is 32 to 128 to 384. Hidden layers use ReLU, dropout is 0.3 and the reconstructed output is linear. The loss is mean squared error between input and reconstruction. Adam uses learning rate 0.001; training is full-batch for 200 epochs. `train_test_split` uses 80 percent train and 20 percent validation with `random_state 42`. With 14,979 usable rows, this means 11,983 training and 2,996 validation rows.

This is unsupervised reconstruction learning. It does not train directly on whether users enjoy an anime. Compressing 384 to 32 reduces representation size by a factor of twelve; that is not a measured twelvefold speedup.

Serving pipeline

Embedding generation reloads the trained model, sets eval mode to disable dropout, extracts 32-dimensional vectors and stores them in `anime_embeddings.pkl`. The application loads 14,976 ID-keyed entries and ranks by NumPy cosine similarity. It excludes the seed item. The multiple-liked-items endpoint averages valid liked vectors to create a taste vector.

The optional item-to-item personalized route multiplies similarity by one plus `genre_boost` divided by ten. It reranks the retrieved candidates; it does not search the full catalog again using the personal signal. The score is a ranking value, not a probability.

Cold start and supporting content

For an unseen anime, metadata can be fetched from AniList. Live sentence-transformer embedding generation is disabled in the service. The route uses genre overlap when metadata provide genres, or returns an error if no usable fallback exists. Upcoming anime and reviews use AniList/Jikan; trailers use YouTube; news uses an RSS feed. Those content integrations do not train the recommendation model.

An artifact compatibility issue

The inspected latent artifact stores genres as a comma-separated string. Some ranking/fallback code iterates genres as if it were a list, which can yield character-level matching. The separate catalog used for explicit likes can have a different representation. Normalize these boundaries before asserting that every personalized route is correct. The current passing tests do not establish full real-artifact genre consistency.

Source: `scripts/train_anime.py`; `scripts/generate_anime_embeddings.py`; `models/anime_dnn.py`; `services/anime_recommender.py`; `routers/anime.py`.

Movie Recommendation Pipeline

Content representation

The movie builder reads SQL catalog rows, combines overview and genres, and fits TF-IDF with English stop words, unigrams and bigrams, min_df 1 and max_features 5,000. The resulting dense matrix is 255 by 5,000. TF-IDF gives more importance to informative terms that are uncommon across the catalog.

For item-to-item recommendations, the service compares a seed movie vector with every catalog vector using cosine similarity and excludes the seed. For multiple liked movies, it averages their vectors. The legacy auto-seed path uses cached personal ratings at least seven with weight rating minus 5.5.

Cosine similarity is dot product divided by the product of vector lengths. It compares direction rather than raw text length. Two movies with related descriptions and genres can therefore rank highly even when their exact words differ.

Current authenticated recommendation route

The router combines explicitly supplied liked IDs with persisted feedback likes. If neither likes nor SQL personal ratings exist, it attempts a cross-domain movie genre profile. If no usable crosswalk candidates exist, it draws from the full catalog using a popularity-oriented fallback. It then reranks with feedback/exposure information, resolves poster URLs, and records shown impressions.

The response source field distinguishes feedback_profile, personal_ratings, crosswalk_profile or sampling_fallback. This source is useful evidence during the demonstration: a nonempty recommendation list alone does not identify how it was generated.

Rating and identity limitations

The local SQL database contains zero personal ratings. The cached movie metadata contain 251 older ratings and must not be presented as fresh per-user behavior. The shared movie signal queries all rated catalog rows without using the supplied user_id, so it is not a true per-user rating store.

The artifact resolver accepts both TMDB IDs and local SQL primary-key strings in one dictionary. These namespaces can collide. The leave-one-out evaluator includes an ambiguity check rather than silently evaluating a different film.

Evaluation options

The reproduced precision/recall/F1 numbers use genre overlap as proxy relevance. A stronger personal test uses genuine IMDb ratings and leave-one-out ranking. scripts/eval_movies_loo.py requires at least twenty liked movies, compares random/popularity baselines and reports HR@10, NDCG@10 and truncated MRR. The current local database cannot satisfy that rating requirement. A correlation below a synthetic-warning threshold does not prove rating authenticity.

Source: scripts/build_movie_tfidf.py; services/movie_recommender.py; routers/movie.py; scripts/eval_movies_loo.py.

Tourism Dining and Fashion Recommendations

Tourist places and restaurants

The current tourism/dining services rank SQL catalog entries by crosswalk-derived category weights. If no crosswalk signal exists, each category receives a uniform base weight. User feedback adjusts a category by two percent per net like, capped at plus/minus ten percent. An explicitly disliked place has its score halved. A hash-derived tie-breaker orders similar scores; Python hashing can vary across processes.

These services do not currently fit TF-IDF, train a neural model, optimize routes or compute nearest-distance ranking. Latitude and longitude are catalog metadata. The README's broad TF-IDF description should not override the actual category-scoring implementation.

Fashion collection and profile

The opt-in Manifest V3 extension parses visible or structured Myntra page content, builds event objects, deduplicates them and queues API delivery. Backend schemas validate events, event IDs provide idempotence, and connection settings control collection. Events and products feed a structured profile of categories, brands, colors, sizes, materials, fits and price range.

Event weight decays as $\text{base_weight} \times \exp(-0.02 \times \text{age_in_days})$. Examples: product view 2, long view 3, wishlist add 7, cart add 10 and purchase 15; wishlist/cart removals contribute negative weights. The profile represents observed activity, and a view is weaker evidence than a purchase.

Two fashion ranking paths

The basic recommender uses category, brand, color and price-profile matching with filters. The richer verdict engine computes direct browsing TF-IDF similarity, a media-to-fashion crosswalk score, price fit and redundancy. Available components are renormalized: direct weight 0.5, crosswalk 0.3 and price 0.2. Redundancy subtracts 0.25 times its value; fit is clipped to zero through one.

BUY requires a percentile of at least 80 and fit at least 0.25; SKIP applies below percentile 40; other scored cases become CONSIDER. Missing fit may produce no verdict. The output is a heuristic preference fit, not a purchase probability or a financial recommendation.

The fashion crosswalk maps media genres to catalog attributes, for example action to black/red/grey and casual items. The source labels this as a hypothesis. The historical evaluation does not demonstrate an improvement from adding it.

Source: services/tourist_spots.py; services/dining_spots.py; services/myntra_profile.py; services/myntra_recommender.py; services/myntra_verdict.py; extension/src.

Cross Domain Transfer Feedback and Convergence

Combining signals

The shared profile collects Spotify genre weights, explicit anime likes, AniList watched-list genres and movie ratings. Anime likes contribute weight 2 per genre. AniList entries with CURRENT, COMPLETED or REPEATING status contribute rating divided by ten times 2, or weight 1 when unrated. Movie ratings contribute rating divided by ten times 2, subject to the shared-rating limitation.

Myntra preferences remain a separate structured namespace. Genre crosswalk dictionaries map music/anime signals into anime genres, movie genres, tourism categories and dining categories. The genre vocabulary and weighting scale must match for transfer to be meaningful.

For example, a rock weight can boost Action/Adventure-style anime according to a hand-authored mapping. This gives a transparent initial ranking hypothesis when the target domain lacks direct feedback. It does not establish that every rock listener likes action anime.

Recommendation feedback and rotation

The shared reranker min-max normalizes base scores, penalizes recent exposures within 24 hours, boosts similarity to liked examples and applies a smaller dislike-similarity penalty. Exact liked and disliked item IDs are excluded from the output. It considers the top 60 adjusted candidates and samples using Gumbel/softmax with temperature 0.15.

MMR then orders the already sampled slate using 0.7 times relevance minus 0.3 times maximum similarity to previously chosen items. This encourages variety earlier in the displayed order. Since it only reorders the sampled slate, it does not itself improve the set's diversity. Randomness also means recommendations can differ between requests.

Feedback updates rankings and profiles; it does not automatically retrain the neural models. Exposure records tell the application what was shown, not what the user liked.

What the convergence number means

Convergence = 40 times active_domains divided by four, plus 60 times average mapped-profile cosine agreement. Coverage counts music, combined anime/AniList, movies and fashion. Agreement currently compares mapped music-to-anime weights with observed anime weights and mapped movie weights with the movie profile. When no eligible pair exists, agreement contributes zero.

It is a zero-to-100 heuristic UI indicator. It is not model accuracy, a confidence probability, measured satisfaction or proof that recommendations are good. A connection can exist without nonzero taste signal. The historical mean of 20 across four users belongs to older saved evidence and is not a current user-study result.

Source: services/taste_profile.py; services/rerank.py; routers/feedback.py; tests/test_convergence.py.

Reproduced Model Evaluation Results

These numbers were reproduced on 6 October 2026 from existing model files and the dense anime cache. The report adapter executed the original numeric evaluation functions, omitted the unused transformer import because the cache existed, and captured confusion matrices without plotting. The full script could not import matplotlib in the application environment. No models were retrained and original evaluation artifacts were preserved.

Evaluation	Precision	Recall	F1	Additional result
Anime genre overlap proxy	0.7316	0.6783	0.7040	MSE 0.001542806; latent std 0.04275724
Movie genre overlap proxy	0.8160	0.6311	0.7117	5,000 TF-IDF features
Spotify DNN genre overlap	Not measured	Not measured	Not measured	No genre labels in track artifact

What the relevance labels are

For anime and movies, the evaluator samples up to 200 query items, selects five similar candidates as predicted positives, and samples five random candidates as predicted negatives. Actual relevance means at least one shared genre. Anime evaluation uses the 20 percent validation partition. Movies use catalog item queries rather than a held-out user-rating dataset.

Confusion matrix count	Anime	Movies
True negative	640	520
False positive	263	184
False negative	340	477
True positive	717	816
Evaluated query candidate pairs	1,960	1,997

Precision = $TP / (TP + FP)$; recall = $TP / (TP + FN)$; F1 is the harmonic mean of those two values. MSE measures reconstruction error, while latent standard deviation checks representation variation. None of these measures is a user satisfaction percentage.

Interpretation limits

The anime evaluator ranks by Euclidean latent distance; serving ranks by cosine similarity. The figures therefore do not directly validate the deployed retrieval rule. Genres are also included in the input texts, so matching genres is an intentionally limited content-consistency proxy.

Random candidates can overlap with retrieved candidates and sampled recall is not full-catalog Recall@K. Nearest-neighbor selection removes a positional candidate rather than explicitly masking the seed ID, which is fragile under ties. Cached feature alignment and model/data provenance should be checked in a full reproducibility study.

Source: `qa/verified_metrics.json`; `qa/verify_metrics.py`; `scripts/evaluate_models.py`.

Historical Results and Missing Quality Evidence

Historical saved evaluation

Component	Saved evidence	What it supports
Tourist places	P/R/F1 = 1.0 on four feedback rows	Very small exploratory check; feedback also influences ranking
Dining	Zero feedback rows	No measurable relevance quality
Generic Myntra evaluator	Reported no events or feedback	Older DB state; current local DB has 29 events
Cross-domain convergence	Four users, all score 20	Heuristic distribution only, not ranking quality
Spotify DNN	Missing genre labels	Current saved evaluation cannot produce relevance metrics

Tourism quality must be tested after removing held-out feedback from ranking. The saved evaluation reuses feedback to adjust categories and then compares recommendations with that same feedback. Dining needs actual user labels before precision/recall can be meaningfully computed; missing evidence is not zero accuracy.

Historical fashion ablation

docs/fashion_verdict_eval.md describes one user, three view-derived positive products and 99 sampled negatives per positive. It is not statistically conclusive.

Variant	HR at 10	MRR	Mean percentile
A Baseline price or popularity	0.0000	0.0173	32.66 percent
B Direct TF-IDF only	1.0000	0.2560	95.29 percent
C Direct price redundancy	0.0000	0.0216	52.53 percent
D Full model with crosswalk	0.0000	0.0157	36.03 percent

The full crosswalk model did not improve these recorded metrics. Direct-only ranked the three examples better. The evaluator excludes held-out events from the direct profile but derives some shared profile/pool components before the holdout, so price/gender/profile leakage needs an audit before stronger claims.

HR@K means the fraction of held-out positives found in the first K positions. MRR is the average reciprocal rank; NDCG rewards positives near the top. These require a declared candidate pool and label definition. A page view is not equivalent to a purchase or an explicit like.

Remaining experiments

Spotify needs a chronological, leakage-free evaluation. Movies need genuine participant ratings and an aligned per-user store. Tourism/dining need held-out feedback. Crosswalks need direct-only versus full-model and shuffled-mapping comparisons. Latency, load, cloud reliability and multi-user satisfaction have not been measured in this report.

Source: data/eval/metrics.json; docs/fashion_verdict_eval.md; scripts/evaluate_fashion_verdict.py; scripts/evaluate_models.py.

Current Software Test Results

The backend suite ran offline against a copy of local_polytaste.db. External socket connections were blocked and loopback was allowed. All 136 tests passed with three cookie-API deprecation warnings. These results concern software behavior under the tests; they are not model accuracy and do not verify live OAuth/provider availability.

Backend test file	Passed	Backend test file	Passed
test_anilist_auth	4	test_anilist_taste	9
test_anime_extensions	23	test_anime_recommender	3
test_content_filtering	3	test_convergence	11
test_eval_movies_loo	6	test_import_imdb_ratings	9
test_movie_rotation_local	1	test_myntra_ingestion	7
test_myntra_models	2	test_myntra_rate_limit	1
test_myntra_schemas	4	test_myntra_seed_balance	7
test_myntra_verdict	11	test_preferences	2
test_recommender	4	test_rerank	3
test_taste_profile	17	test_tourist_spots	9

Every listed file had zero failures, errors and skips in this run. The JUnit XML retains individual test names. The older root pytest.log contains failures from a previous version and should not be cited as the current result.

What was checked

The suite covers mocked Spotify scoring and API handling; anime metadata/content fallbacks; AniList auth and taste signals; adult-content filters; shared profiles and convergence; IMDb import and movie-evaluator guards; local movie rotation and feedback; fashion schemas, persistence, replay/idempotency, connection controls, rate limiting and verdict logic; preferences; reranking; and tourism feedback/ranking.

One anime precomputed-embedding test ends with pass without an assertion. This contributes to the test count but does not prove the embedding is valid. No dedicated Spotify history-import test file is present. Passing assertions should be judged for their actual scope.

Extension and frontend

All 17 Node extension tests passed across collection-parser, product-parser, queue, search-listing-parser and verdict-badge. The verdict-badge file was included explicitly because the package test command omits it. These use fixtures or simulated DOM behavior, not a live Myntra browsing session.

TypeScript checking and the Vite production build passed. Vite warned about a JavaScript chunk over 500 kB; the reported main chunk was about 650.48 kB before gzip. The build checks syntax/types/bundling, not every interactive user flow. No browser end-to-end test was run here.

Source: qa/pytest_results.xml; qa/current_pytest.log; qa/extension_tests.log; qa/frontend_typecheck.log; qa/frontend_build.log.

Demonstration Plan and Honest Claims

Suggested ten minute demonstration

1. Introduce the problem and architecture, then identify the individual recommendation methods.
2. Show the Audio JSON format and aggregate count of 28,407 records. Use an anonymized example and explain that it represents one participant.
3. Show the local data catalogs and database counts. Keep the older DB and cached ratings clearly separated.
4. Sign in with the intended Google flow and show optional connections if credentials and callbacks are ready. Verify the backend response before describing a connected profile.
5. Demonstrate one anime seed, movie recommendations and the movie response source. Show a feedback action and how a later recommendation slate changes.
6. Show tourism/dining category ranking and explain one crosswalk example. If showing fashion, identify whether it uses seeded products, observed events or a live extension capture.
7. Present the 136 backend and 17 extension test passes, then the proxy evaluation table and its limitations. End with the next experiments required for stronger claims.

Checks before the presentation

Use the same backend/database throughout. The documented local addresses are 127.0.0.1:8000 for FastAPI and 127.0.0.1:5173 for Vite. The browser API base and cookie host must match. Do not assume localhost and 127.0.0.1 share cookies. Keep the test logs and explanation available if API access fails.

The dashboard has mock-data fallbacks, and main.py includes empty generic activity/recommendation placeholder endpoints. Inspect actual network responses/source fields before presenting a UI row as personalized output. Importing a friend's history into a profile should use the correct participant/demo identity and should not overwrite someone else's profile.

Security and deployment limits

Google OAuth and signed HttpOnly cookies are implemented, but the alternate /auth/login currently accepts any nonempty email/password and signs that email. The session serializer does not enforce a server-side age limit. Several debug routes expose user/connection diagnostics; token storage is not application-encrypted. Folder import has a user override and accepts a server-side path. These are prototype limitations requiring hardening before deployment.

The code supports PostgreSQL and includes Docker and Render configuration. README describes Azure and docs/architecture.md describes an older AWS design with Amazon. The current mounted source does not prove which service is live. Describe configuration capability separately from verified deployment; no AWS GNN/DynamoDB architecture should be claimed from the stale document.

A production extension needs explicit consent, data minimization, robust deletion/export and provider-safe collection. Live API behavior, deployment security and performance were not exercised by this offline report.

Source: LOCAL_SETUP.md; main.py; services/auth.py; routers/spotify_import.py; render.yaml; frontend dashboard code.

Viva Questions About the Design

1 What problem are you solving

Several services each know only part of a user's preferences. PolyTaste brings those signals into one application and explores using preferences from one domain to initialize another domain's recommendations.

2 What is your main contribution

The implemented contribution is the end-to-end integration of independent domain recommenders, shared taste profiles, explainable crosswalk rules, an opt-in fashion extension and feedback-based ranking. The crosswalk's predictive benefit remains an evaluation question.

3 Which GNNs did you use

None in the current implementation. Anime uses a feed-forward autoencoder and Spotify has a feed-forward pair-similarity DNN. A GNN would need explicit graph data and message-passing layers.

4 Is the system content based or collaborative

It is primarily content-based and rule-based, with a Spotify session co-occurrence experiment. It has no demonstrated multi-user collaborative filtering model.

5 Why choose different algorithms for different domains

Each domain has different data. Anime has rich text suitable for embeddings, movies have small text catalogs, places have category metadata and fashion has product attributes/events. The implementation follows the available representation rather than forcing one model on every domain.

6 What is an embedding

An embedding is a numeric vector representing an item. Here it represents anime text or Spotify session relationships, letting numerical similarity compare items.

7 What is an autoencoder

It encodes a vector into a smaller latent vector, then decodes it back. Training minimizes reconstruction error. The latent vector is subsequently used for recommendation similarity.

8 Why compress anime embeddings

The model reduces 384 values to 32 for storage and vector comparisons. Whether that improves speed and preserves quality must be measured against uncompressed embeddings.

9 Why use cosine similarity

It compares vector direction after accounting for magnitude. It is useful for text/embedding similarity, though the best similarity rule should be checked empirically.

10 What happens to a new user

Catalog/popularity or uniform-category fallbacks can provide initial items, and connected source domains can supply crosswalk weights. These initial recommendations have weaker evidence than personalized history.

11 What happens to an unseen anime

The service fetches metadata but live embedding generation is disabled. It can use genre overlap; missing metadata prevents that fallback.

Viva Questions About Data and Implementation

12 Is your Spotify data real

Yes, the supplied export records one friend's real listening behavior: 28,407 Audio records, of which 28,336 have the metadata retained by the current filters. It is one participant, not thousands of independent users.

13 What format should the Spotify data use

The original JSON arrays in Streaming_History_Audio_*.json files. The current code reads those names directly. A normalized CSV is optional for research inspection, not required for import.

14 Does Spotify JSON contain genres or audio features

The fields used here are timestamps, playback duration, track/artist names, track URI and skip flag. Artist genres are added through API enrichment. The DNN history path uses co-occurrence/SVD, not measured danceability or energy from this export.

15 How do you interpret skipped playback

The importer counts an explicit skip or playback under 30 seconds as a skip. Its completion_rate is a non-skip ratio, not actual listened duration divided by track duration.

16 Why can many events still be a small study

Repeated listens from one person are correlated. They give detailed evidence about that person's activity but do not establish performance across different users.

17 Where is the database and why is JSON still used

SQL stores accounts, histories and feedback. JSON files are portable catalog or export formats; some structured fields are serialized as JSON text within SQL. Precomputed vectors live in pickle/model files.

18 Can you claim all movie ratings are genuine

No. The code includes a synthetic generator, local SQL has no personal ratings and cached artifacts contain older ratings without a verified provenance manifest. Genuine IMDb imports must be traced to a real export.

19 Does feedback retrain the neural network

No automatic retraining is implemented in the feedback route. Feedback changes stored preferences, exclusions and ranking adjustments. Neural retraining requires the offline scripts.

20 What is a crosswalk

A manually written mapping between genre/category vocabularies. It lets one domain supply an initial preference hypothesis to another. It is explainable but can be wrong or overly stereotyped.

21 What database design would you improve first

Use one consistent internal user key and separate per-user movie ratings from the shared catalog. Audit the tourist/dining foreign keys and item-ID namespaces for relational consistency.

22 Which deployment have you verified

This inspection verified local source, tests and artifacts. It found PostgreSQL support plus Docker/Render configuration and historical Azure/AWS descriptions, but did not verify a currently running cloud deployment.

Viva Questions About Results and Limitations

23 What test results can you show now

The offline backend run passed 136 tests, all 17 extension tests passed, and TypeScript/Vite build checks passed. Anime/movie proxy metrics were reproduced from cached local artifacts. Live OAuth and browser capture were not tested in that run.

24 Does 136 passed mean 100 percent recommendation accuracy

No. It means the existing test assertions passed in that environment. Recommendation quality is a separate measurement requiring relevance labels, a declared candidate pool and a held-out protocol.

25 What do your anime and movie metrics measure

Whether sampled candidate pairs share genres under the existing evaluator. Anime precision is 0.7316 and movie precision is 0.8160. These are genre-consistency proxy results, not measured personal enjoyment.

26 Why not report Spotify DNN test accuracy

The trainer uses all pairs for training and has no held-out test split. The saved evaluator lacks genre labels. I can explain the model, but I cannot honestly claim a measured generalization score.

27 What does the anime MSE prove

It quantifies reconstruction error for held-out dense input vectors. It does not prove the top recommendations satisfy users, and it must be considered alongside retrieval metrics and baselines.

28 Why is the tourist score of one weak evidence

It used only four historical feedback rows and reused feedback in ranking. The sample is tiny and the holdout is not independent. It is exploratory evidence, not reliable perfect accuracy.

29 Did the fashion crosswalk improve ranking

The historical three-positive experiment did not show that: the full variant had MRR 0.0157 compared with 0.0216 for the variant without crosswalk. Direct-only performed better on those examples. A larger leakage-free test is needed.

30 What is data leakage and how would you avoid it

Leakage happens when test information influences features, profiles or model selection. For Spotify, split sessions before SVD/co-occurrence. For feedback evaluations, remove held-out items from every profile component before ranking.

31 What does convergence of 80 mean

It is a heuristic mixture of source coverage and mapped-profile agreement, bounded between zero and 100. It does not mean 80 percent accuracy or an 80 percent chance of liking a recommendation.

32 What is your next improvement

First fix data representation/user-ID consistency and gather verified per-user labels. Then compare direct-only and crosswalk models using chronological holdouts and identical baselines. A graph model is a possible later experiment after suitable multi-user interaction data exist.

Source Map and Report Generation Prompts

Files to open when explaining the code

Topic	Main source files
API assembly and middleware	main.py
SQL entities and sessions	database.py and models/myntra.py
Google session and OAuth	services/auth.py and routers/google_auth.py
Spotify live scoring and sync	routers/spotify.py and services/spotify_sync.py
History profile import	scripts/import_spotify_history.py and routers/spotify_import.py
Spotify neural experiment	scripts/train_spotify_history.py and models/spotify_dnn.py
Anime training and serving	scripts/train_anime.py and services/anime_recommender.py
Movies	scripts/build_movie_tfidf.py and services/movie_recommender.py
Tourism and dining	services/tourist_spots.py and services/dining_spots.py
Shared taste and ranking	services/taste_profile.py and services/rerank.py
Fashion verdict	services/myntra_profile.py and services/myntra_verdict.py
Extension collection	extension/src/content and extension/src/storage
Current tests and reproduced metrics	docs/professor_pack/qa

Prompt for the formal project report

Open Report_Generation_Prompt.txt and supply the repository plus this explanation and QA evidence. It asks for a full academic report, architecture/data-flow diagrams, database design, per-component algorithms, measured results, limitations, sources and viva questions. It explicitly prohibits invented GNN results or deployment claims.

Prompt for test results

Open Test_Results_Prompt.txt in the coding environment. It asks the agent to run feasible checks, preserve databases/models, report every test module, reproduce existing metrics, and design independent model and crosswalk experiments. New model training requires separate authorization, and missing quality metrics remain Not measured.

A useful short instruction is: "Evaluate every implemented component from the actual code. Separate software tests, proxy evaluations and held-out user-quality tests. Use local copies, state splits and baselines, include failures and blocked measurements, and never invent GNN or accuracy results."

Evidence files to preserve

qa/inventory.json records counts and artifact hashes. qa/verified_metrics.json records reproduced numbers and confusion matrices. qa/pytest_results.xml records every backend test case. The current_pytest, extension_tests, frontend_typecheck and frontend_build logs record the observed check outputs. qa/verify_metrics.py documents the numerical adapter.

The source Markdown is editable and the Word version presents the same explanation. Use the full prompts as attachments rather than asking a report generator to rely on a screenshot or undocumented metric table.